MONITORIA TI APRESENTA

LÓGICA DE PROGRAMAÇÃO EM JAVA

06

AULA 06 :

LÓGICA DE PROGRAMAÇÃO EM JAVA

PONTOS CHAVE



- Revisão
- Resolução de problemas
- Teste e debug
- Avaliar problemas

INPUT E OUTPUT



- Scanner lê inputs no terminal
- System.out.print exibe outputs no terminal

```
System.out.print();
```

```
import java.util.Scanner;

public class Main {
    static Scanner sc = new Scanner(System.in);
```

VARIÁVEIS



- int
- String
- double
- boolean

```
int inteiro;
String texto;
double numero_com_virgula;
boolean verdade_ou_falso;
```

OPERAÇÕES



- +
- -
- *
- /
- %

DECISÕES E REPETIÇÕES



- **Decisões:** permitem executar diferentes ações de acordo com uma condição.
- Utilizam estruturas como **if** e **else**.

- **Repetições:** permitem executar um trecho de código várias vezes.
- Utilizam estruturas como for e **while**.

MÉTODOS E VETORES



- **Métodos: organizam o programa em partes menores.**
- **Vetores: armazenam vários valores do mesmo tipo em uma única estrutura, utilizando índices que começam em 0.**

EXEMPLO

```java
public class VetoresMetodos {
    Run | Debug
    public static void main(String[] args) {
        int[] notas = {7, 8, 9, 6, 10};
        // chamada
        int r = soma(notas);
        /* o método recebe a referência: alterações valem também para
        o array original */

        System.out.println(r); // 40
    }

    public static int soma(int[] vet) {
        int total = 0;
        for (int x : vet) total += x;
        return total;
    }
}
```

INTERPRETAÇÃO



É importante compreender o problema e identificar:

- **Objetivo:** o que o programa precisa fazer?
- **Entrada:** quais informações serão recebidas?
- **Processamento:** quais operações serão realizadas?
- **Saída:** qual resultado será apresentado?

DIVIDINDO UM PROBLEMA



Um problema grande pode ser dividido em tarefas menores para facilitar sua compreensão e resolução.

TESTES DE UM PROGRAMA



- **Casos normais:** entradas comuns e esperadas.
- **Casos-limite:** valores próximos aos limites definidos pelo problema.
- **Entradas inválidas:** dados que não atendem às regras estabelecidas.

ENTENDENDO ERROS



Para investigar um erro:

- **Leia a mensagem apresentada.**
- **Identifique o tipo de erro.**
- **Procure o número da linha indicada.**
- **Verifique o código próximo à linha.**
- **Corrija o problema e execute novamente.**

EXPLICANDO UMA SOLUÇÃO



Ao apresentar um programa, é importante explicar:

- **O que ele faz:** qual problema resolve.
- **Como funciona:** quais etapas executa.
- **Por que foi feito assim:** quais estruturas e recursos foram escolhidos.

EXERCÍCIOS:

Façam um sorting de um array de inteiros com 10 espaços, o usuário precisa ser capaz de dar input de todos os valores do array e depois disso os valores devem aparecer em ordem crescente no terminal

Algorítmos conhecidos: bubble sort, quick sort, heap sorte, miracle sort...



PERGUNTAS? COMENTÁRIOS?

Fique à vontade para compartilhar sua opinião.

